

Advanced Programming Techniques

fbeit

FACHBEREICH ELEKTROTECHNIK
UND INFORMATIONSTECHNIK

Prof. Dr. Michael Lipp

fbeit

FACHBEREICH ELEKTROTECHNIK
UND INFORMATIONSTECHNIK

C++ Libraries

Boost Socket

More Libraries

- The C++ Standard library does not cover everything
- Notably no classes for network connections
- Additional libraries are available
 - e.g. <https://github.com/fffaraz/awesome-cpp>
- Examples
 - Boost: well known, large, covers a lot of topics
 - Poco: focused on developing network applications
 - FLTK: GUI library

Adding a library

- **Structure**

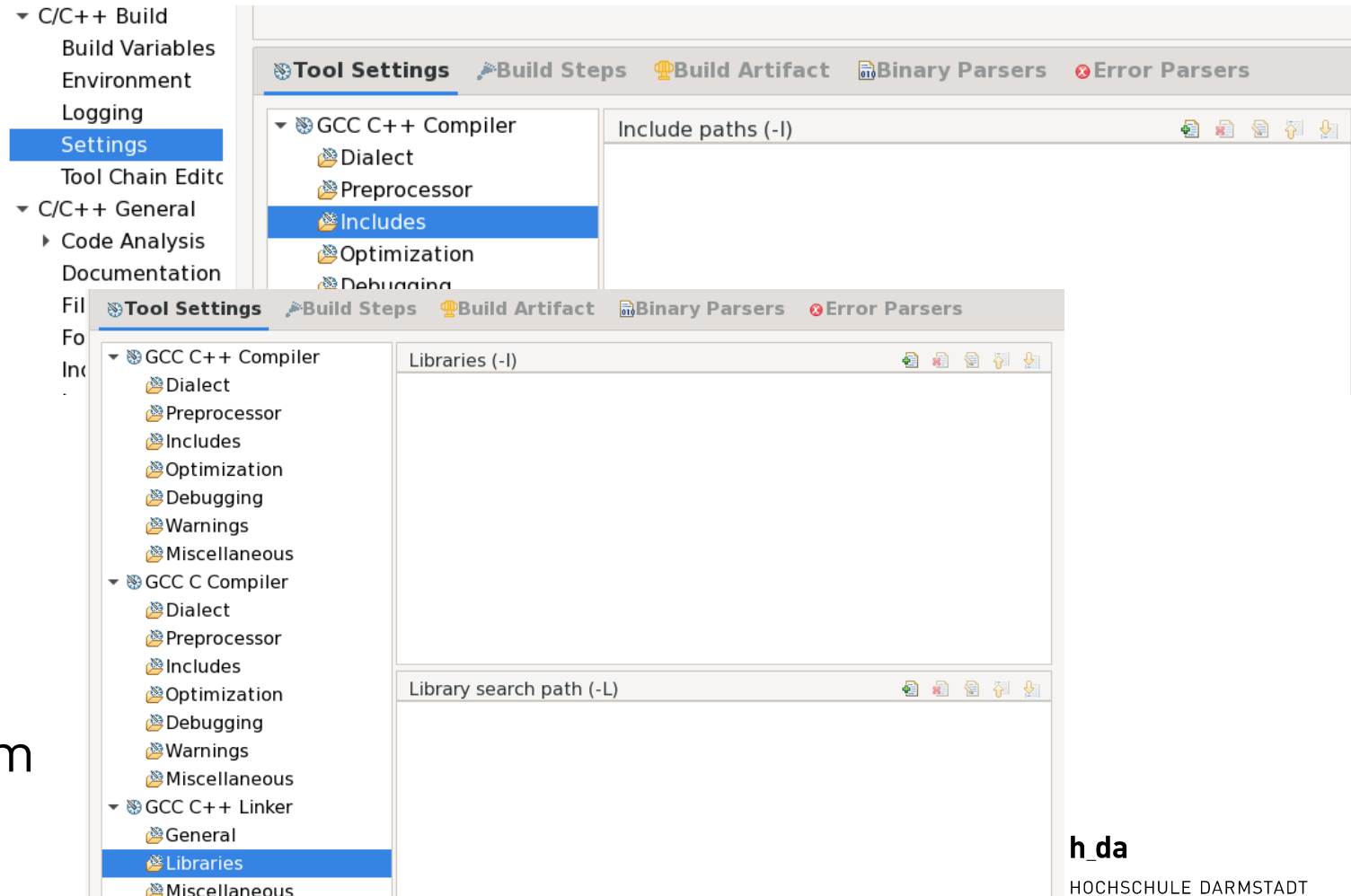
- A library consists of
 - header files
 - precompiled code (a.k.a “library”)

- **Adding to project**

- Copy all files into project
- Configure in Eclipse

- **Use “packaged” library**

- Install in Operating system
- Configure in Eclipse



Packaged library

- **Install prepackaged library**

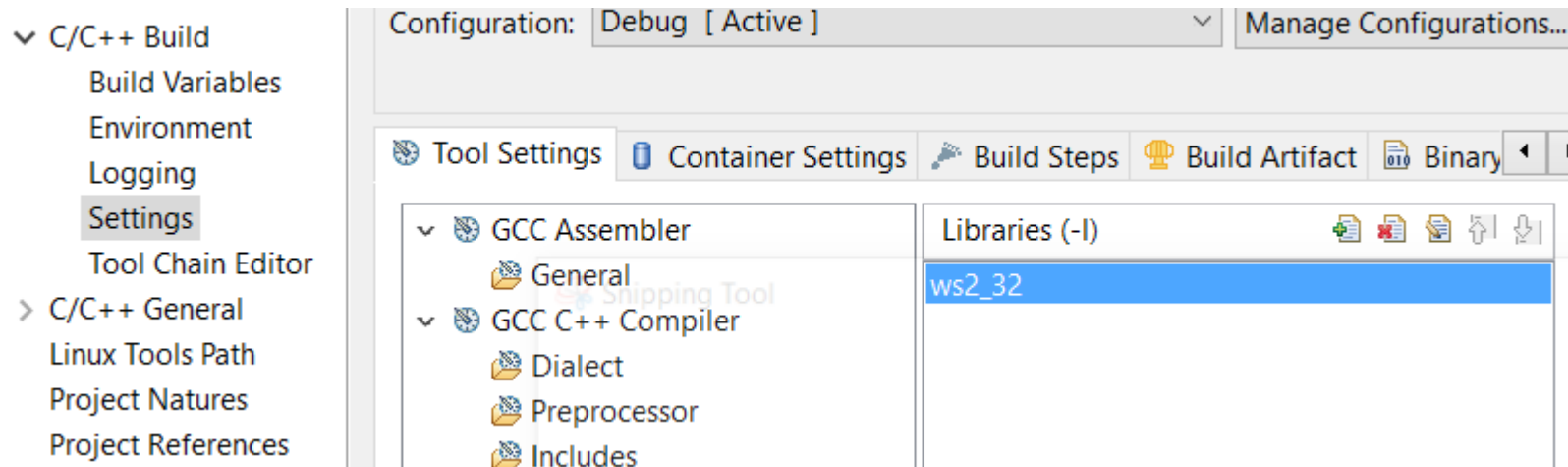
- In File-Explorer navigate to `C:\Program Files\GIT\eclipse\mingw`
- Right-click on `msys2.exe` and “Run as Administrator”
- In window, type
 - `pacman -S mingw-w64-x86_64-boost`
 - ... be patient
 - (For the Poco library, type “`pacman -S mingw-w64-x86_64-poco`”)

- **Result**

- Header files are added to “system” header files, i.e. can simply be `#include-ed`
- Libraries can be added by name (next slide)

Packaged library

- Add the library



- In this case the general Windows network library is added, no special boost library required

Networking

- Dominated today by the Internet-Protocol (IPv4, IPv6)
- Basic concepts
 - Network participants (“Computers”) have IP-addresses
 - Example IPv4: 172.16.254.1 (32 bits denoted as decimal values of four 8-bit groups)
 - Example IPv6: 2001:0db8:85a3:0000:0000:8a2e:0370:7334 (128 bits denoted as hex values of 8 groups with 16 bits each)
 - Luckily a service (Domain Name Service) maps human readable names (e.g. www.h-da.de) to IP-addresses
 - Data is exchanged between communication endpoints
 - An endpoint consists of an IP-address and a port number
 - Servers “listen” for connections, clients connect to servers
 - A connection is a pair of two end points

Simple service example

- **Daytime Service (RFC 867)**
 - Listens on port 13
 - Upon connection, returns string with time information
- **Public servers**
 - `time-a-g.nist.gov`
 - `time-b-g.nist.gov`
 - `time-c-g.nist.gov`
 - `time-c-g.nist.gov`

Boost socket

- Part of the Boost Asio (asynchronous i/o library)
 - https://www.boost.org/doc/libs/1_70_0/doc/html/boost_asio/reference.html
- Communication endpoint modeled as “socket iostream”
- Boost likes namespaces
 - Full class name: `boost::asio::ip::tcp::iostream`
- Socket can be created and then connected or connected upon creation
- After successful connection, socket can be used like an **`std::iostream`**

Aside: namespaces

- **C++ namespaces**
 - Often used, never explained
 - A namespace is a scope in which names have to be unique
 - Example:
 - In Germany street names have to be unique within a town
 - Streets with the same name may exist in different towns
 - → A town is a namespace for street names
 - The standard library uses namespace `std` for everything
 - We have used the global namespace for our classes

Aside: namespaces

- “Putting” (defining) something in a namespace

```
namespace myNameSpace {  
  
    class Test {  
    };  
  
}
```

- Referencing something from a namespace

- With prefix: `myNameSpace::Test`
- With using: `using namespace myNameSpace`

- Rules

- Never use `using` in header files (you cannot “undo” it)
- Avoid using `using` in Implementation files (prefix indicates origin)

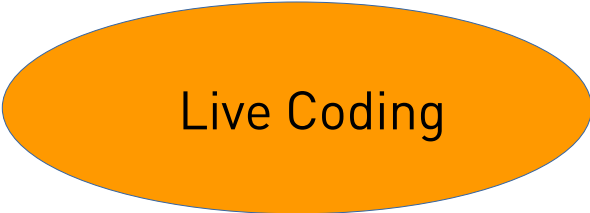
Time for machine processing

- Time Protocol (RFC 868)

- Have a look: <https://tools.ietf.org/html/rfc868>

- Steps

- Create connection
 - Convert 4 bytes to 32 integer
 - Convert to “time”

An orange oval button with a thin black border, containing the text "Live Coding" in black font.

Live Coding

Working with time/date is difficult

- How can we model date/time
 - Class with data members day, month, year, hour minute?
 - Many items
 - Uses unnecessary amount of storage space
 - Calculating duration is a nightmare
 - How many seconds from 24.01.1990 13:42 to 04.12.2019 17:45?
 - The Unix way: Seconds since 1.1.1970 0:00 UTC (“the epoch”)
 - Hard to understand for humans
 - Conversion to “readable” time isn’t easy
 - Seconds: `secondsSince % 60`, minutes: `secondsSince / 60 % 60`, hours: `secondsSince / 3600 % 24`, day of month: ... now it gets difficult
 - → we need another library
 - Be aware of range, or “Reboot your dreamliner” (<https://arstechnica.com/information-technology/2015/05/boeing-787-dreamliners-contain-a-potentially-catastrophic-software-bug/>)

C++11 chrono (originally from Boost)

- **Clocks**
 - Used to obtain absolute or relative time
 - `system_clock`
 - `steady_clock`
 - `high_resolution_clock`
- **Time point**
 - Duration of time that has passed since epoch
- **Duration**
 - Span of time with unit
- **Plus support from C library (`std::time_t`)**

`std::time` and related functions

- **`std::time_t`** is used to represent a point in time
 - Implemented almost always as an integral type that holds the number of seconds since 00:00, Jan 1 1970 UTC
 - Can be converted to `std::tm` which represents the time “as we know it” (day, month, year, hour, minute, second), either by `localtime` or `gmtime`
 - `std::tm` can be converted to `std::time_t` using `std::mktime`
 - Can be converted to human readable form using `std::put_time`
 - See code examples in cppreference.com

Date, Time, DateTime from Poco

- **Timestamp** represents the monotonic time
 - Use TimeVal (64-bit integer) for space efficient storage
- **DateTime** allows access to hour, minute, second, day-of-week, day-of-month, month, year based on the Gregorian calendar
- **Date** and **Time** are wrappers that represent only the parts
- Documentation online, start with
<https://pocoproject.org/docs/Poco.DateTime.html>

Date, Time, DateTime from Poco

- Poco usage

- Requires the addition of libraries for the Poco package used
- Add libraries as shown in the previous slides
- Using date/time requires “PocoData”
- Using networking (upcoming lecture) requires “PocoNet”
- Using multi-threading (upcoming lecture) requires “pthread”

